Data Base Management Tutorial

How to use this tutorial

Try each exercise yourself first, then compare with the correction. Every example builds its own data in memory, so you need no external files. All solutions were run and the outputs below are the real results (with pandas 3.x; on older versions a text column may display as `object` instead of `str`).

```
import io
import numpy as np
import pandas as pd
```

Part 1 – Loading data

Exercise 1 – Read a messy CSV

Here is a CSV text with a comment line at the top, missing values written as `NA` or left empty, and a date column:

```
raw = """# sales report
id,city,amount,date
1,Yaounde,1200,2026-01-05
2,Douala,NA,2026-01-06
3,Yaounde,800,2026-02-10
4,Bafoussam,,2026-02-11
5,Douala,1500,2026-03-01
"""
```

Tasks

1. Load it into a DataFrame, skipping the comment line.
2. Use `id` as the index.
3. Parse `date` as a real date.
4. Count the missing values per column.

Exercise 2 – Files without header, and big files

```
txt = "1,2,3,4,hello\n5,6,7,8,world\n9,10,11,12,foo\n"
```

Tasks

1. Read `txt` with no header and name the columns `a, b, c, d, message`.
2. Make `message` the index.
3. Explain how you would read only the first 2 rows, or process a huge file piece by piece.

Exercise 3 – Writing and binary formats

Task: save the DataFrame from Exercise 1 to CSV and to Parquet, then read both back.

Part 2 – Cleaning and transforming

We reuse `df` from Exercise 1 (with `amount` missing in rows 2 and 4).

Exercise 4 – Handling missing data

Tasks

1. Remove the rows where `amount` is missing.
2. Replace missing amounts by the overall median.
3. Replace each missing amount by the mean of its own city.

Exercise 5 – Duplicates and discretization

```
s = pd.DataFrame({
    "name": ["ann", "bob", "ann", "cy"],
    "age":  [5, 17, 5, 40],
})
```

Tasks

1. Remove duplicate rows.
2. Add a column `group` with the categories child (0–12), teen (13–17), adult (18+).

Exercise 6 – String manipulation

```
names = pd.Series(["  Paul Biya ", "marie NGONO", None, "jean-pierre kamga"])
```

Tasks

1. Remove extra spaces and put each name in title case.
2. Extract the first word of each name.
3. Find which names contain "ngo", ignoring case, without crashing on the missing value.

Part 3 – Combining and reshaping

Exercise 7 – Merge (joins)

```
cust = pd.DataFrame({"cid": [1, 2, 3], "name": ["A", "B", "C"]})
orders = pd.DataFrame({"oid": [10, 11, 12, 13],
                       "cid": [1, 1, 3, 4],
                       "total": [50, 70, 20, 90]})
```

Tasks

1. Keep only customers who have orders.
2. Keep all customers, even without orders.
3. Keep everything and find which rows have no match on the other side.

Exercise 8 – Concatenate

```
a = pd.DataFrame({"x": [1, 2]})
b = pd.DataFrame({"x": [3, 4]})
```

Task: stack `a` and `b` vertically with a clean index `0, 1, 2, 3`.

Exercise 9 – Pivot and melt

```
long = pd.DataFrame({
    "city":  ["Y", "Y", "D", "D"],
    "month": ["Jan", "Feb", "Jan", "Feb"],
    "sales": [10, 20, 30, 40],
})
```

Tasks

1. Turn it into a wide table: one row per city, one column per month.
2. Convert the wide table back to long format.

Part 4 – Group operations

```
g = pd.DataFrame({
    "dept":   ["IT", "IT", "HR", "HR", "HR"],
    "sex":    ["F", "M", "F", "F", "M"],
    "salary": [500, 600, 400, 450, 420],
})
```

Exercise 10 – Aggregation

Tasks

1. Mean salary per department.
2. Count, mean and max of salary per department in one table.
3. Total salary per department and sex, with sex as columns.
4. Named aggregations: `avg` (mean salary) and `n` (number of rows) per department.
5. Range (max − min) of salary per department with your own function.

Exercise 11 – Transform, pivot table and cross-tabulation

Tasks

1. Add a column `dev` giving each salary minus the mean salary of its department.
2. Build a pivot table of mean salary with `dept` as rows and `sex` as columns.
3. Count employees per department and sex.
4. Get the best paid employee of each department.

Mini test (answers at the end)

1. Which argument tells `read_csv` that the file has no header row?
2. What is the difference between `dropna()` and `fillna()`?
3. Which `how=` value of `merge` keeps rows from both tables even when there is no match?
4. What does `transform` return that `agg` does not?
5. Why does `pd.concat([a, b])` sometimes give duplicate index labels, and how do you avoid it?

What to practise next

- Take any CSV of your own (grades, sales, expenses) and repeat Exercises 1, 4 and 10 on it.

- Section 10.2 of your PDF covers column-wise application of several functions at once, a good follow-up to Exercise 10.
- Chapter 9 (matplotlib) is not covered here. I can write a plotting tutorial with corrections in the same format if you want it.